

AI ASSISTED CODING ASSIGNMENT – 10.1

CH. Nakshatra (2303A51094)

BATCH-02

Problem 1: Syntax and Logic Errors

Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input:

```
# Calculate average score of a student

def calc_average(marks):

    total = 0

    for m in marks:

        total += m

    average = total / len(marks)

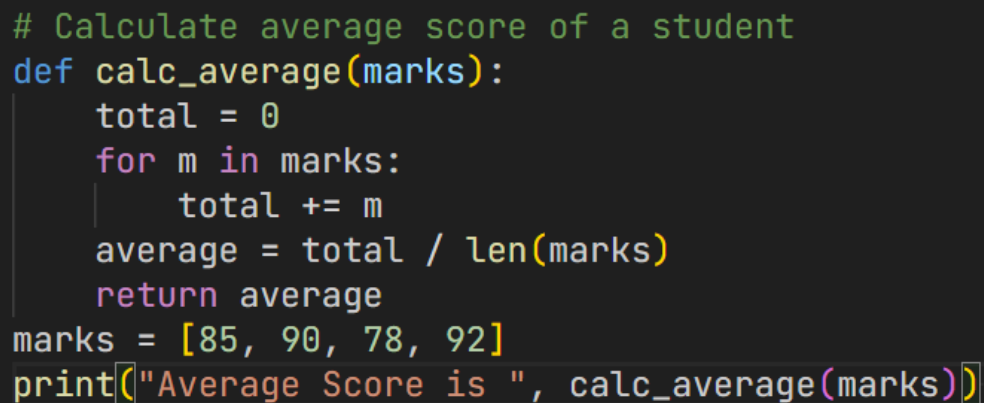
    return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the Fixes.



```
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Problem 2: PEP 8 Compliance

Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input:

```
def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code

```
def area_of_rectangle(length: float, width: float) → float:
    """
    return the area of a rectangle

    Parameters:
    length (float): the length of the rectangle
    width (float): the width of the rectangle

    Returns:
    float: the area of the rectangle
    """
    return length * width
print(area_of_rectangle(5.0, 3.0))
```

Problem 3: Readability Enhancement

Use AI to make code more readable without changing its logic.

Sample Input:

```
def c(x,y):
    return x*y/100

a=200
b=15

print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

```
def calculate_percentage(x: float, y: float) → float:
    """
    Returns the percentage of x with respect to y.

    Parameters:
    x (float): The part value.
    y (float): The total value.

    Returns:
    float: The percentage of x with respect to y.
    """
    return x*y/100
a=200
b=25
print(calculate_percentage(a, b))
```

Problem 4: Refactoring for Maintainability

Use AI to break repetitive or long code into reusable functions

Sample Input:

```
students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

```
def welcome_student(name: str) → str:
    """
    Prints a welcome message for the student.

    Parameters:
    name (str): The name of the student.
    """
    print("Welcome", name)
name = ["Alice", "Bob", "Charlie"]
for student in name:
    welcome_student(student)
```

Problem 5: Performance Optimization

Use AI to make the code run faster

Sample Input:

```
# Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

    squares.append(n**2)

print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

```

#using list comprehension
nums = [i for i in range(1,1000000)]
squares = [n**2 for n in nums]
print(len(squares))

#using vectorization operations
import numpy as np
nums = np.arange(1,1000000)
squares = nums**2
print(len(squares))

```

Problem 6: Complexity Reduction

Use AI to simplify overly complex logic.

Sample Input:

```

def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"

```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

grades / Python / AIXC / demo13.py / grade

#Simplified using elif statements

```
def grade(score):  
    if score ≥ 90:  
        return "A"  
    elif score ≥ 80:  
        return "B"  
    elif score ≥ 70:  
        return "C"  
    elif score ≥ 60:  
        return "D"  
    else:  
        return "F"
```

#using dictionary lookup

```
def grade(score):  
    grades = {  
        range(90, 101): "A",  
        range(80, 90): "B",  
        range(70, 80): "C",  
        range(60, 70): "D",  
    }  
    for (low,high), letter in grades.items():  
        if score in range(low, high):  
            return letter
```